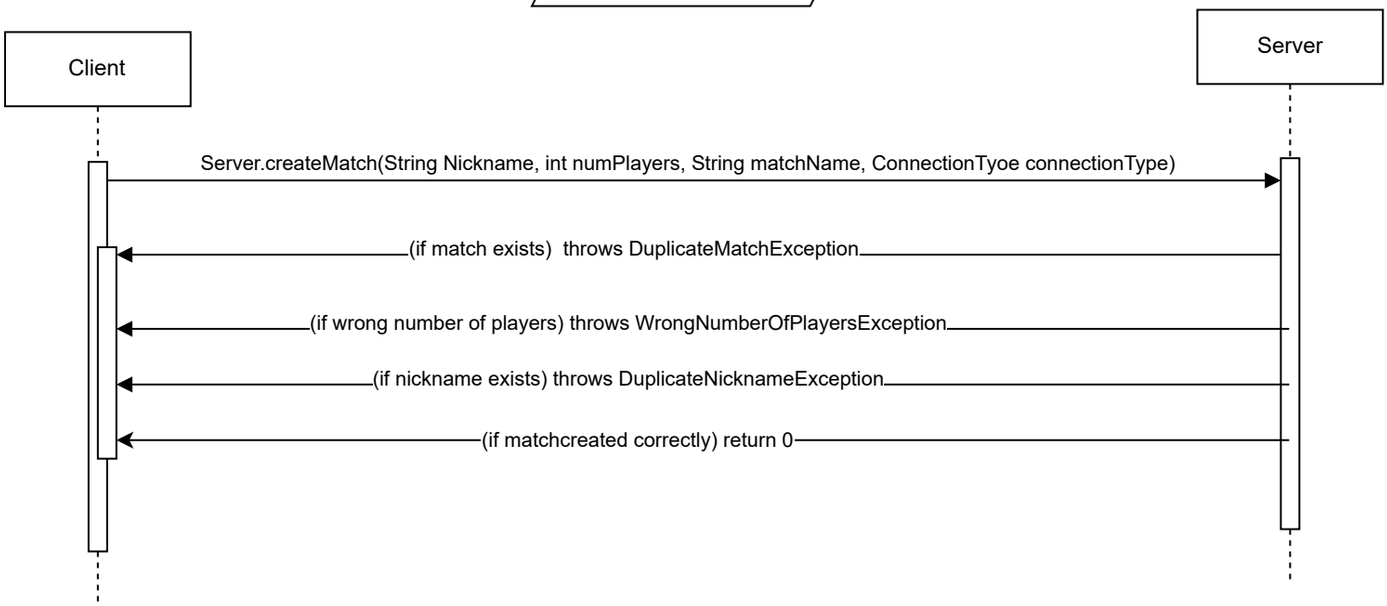
RMI

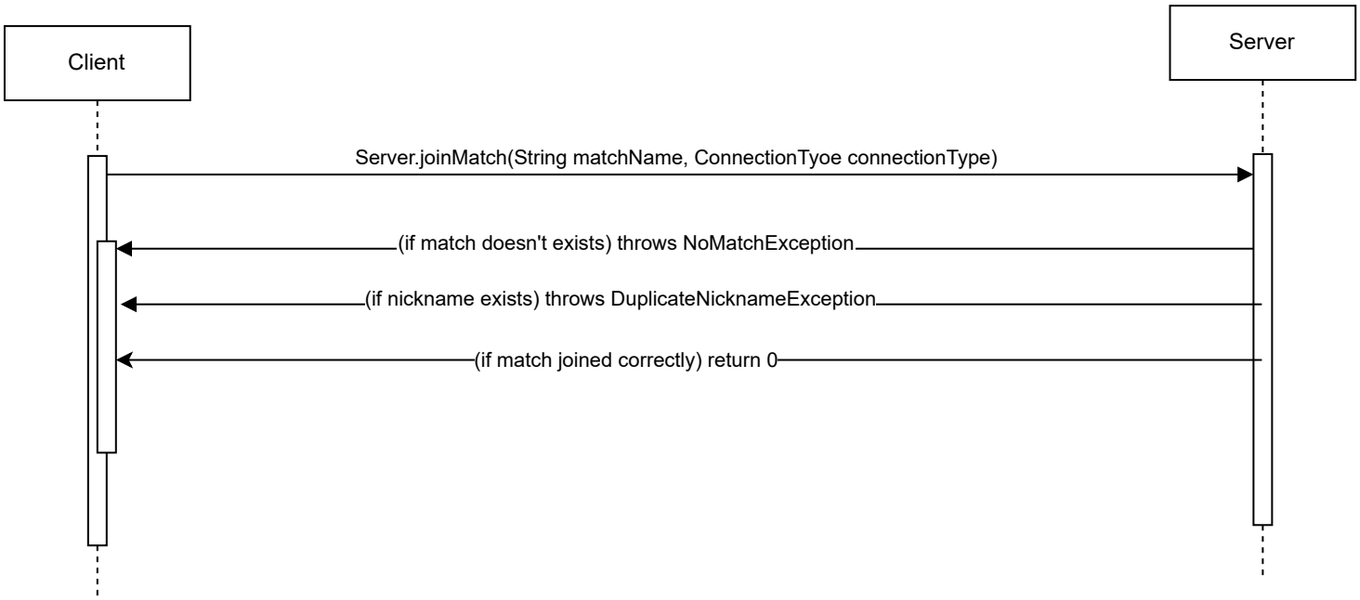
Phase 1: Initialization

Sul RMI registry è definito già acceso il server tramite nome "CodexServer"

Case creation match

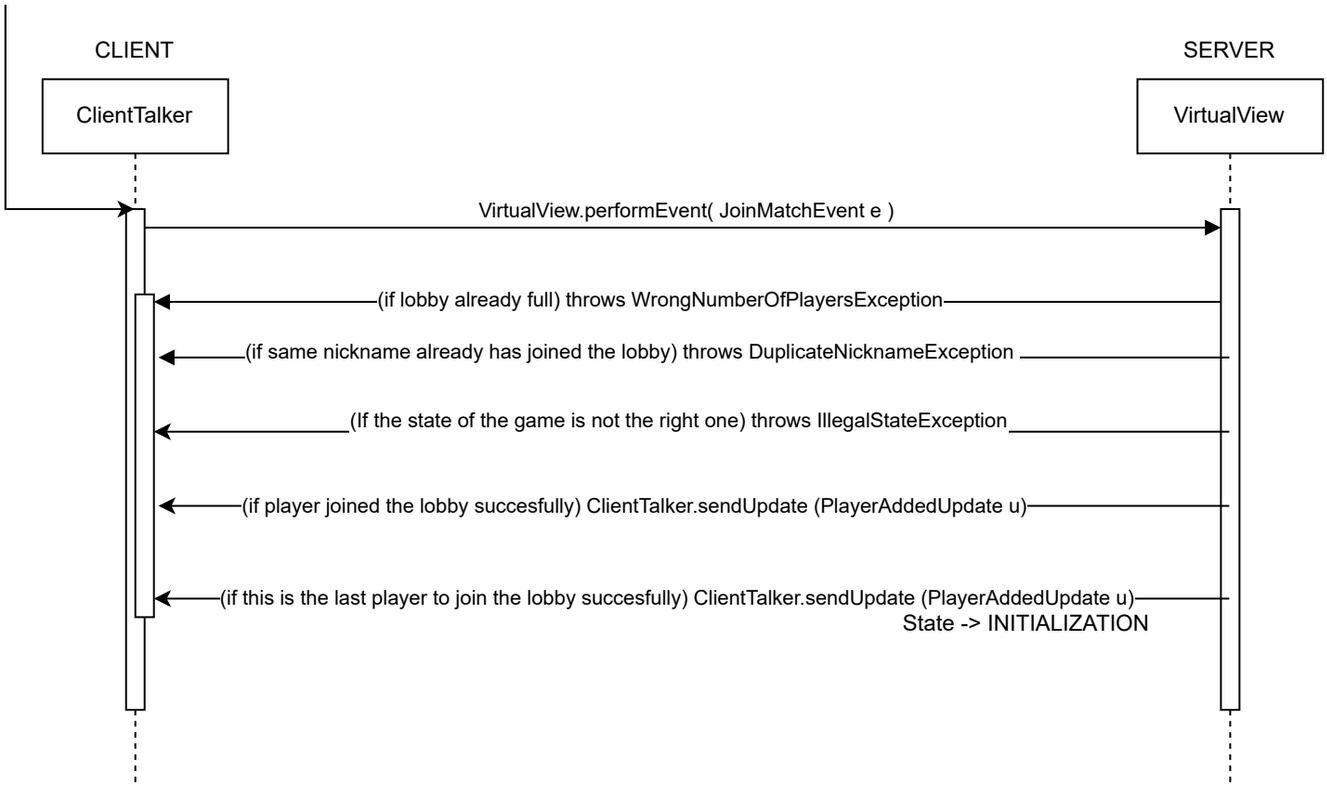


Case Join Match



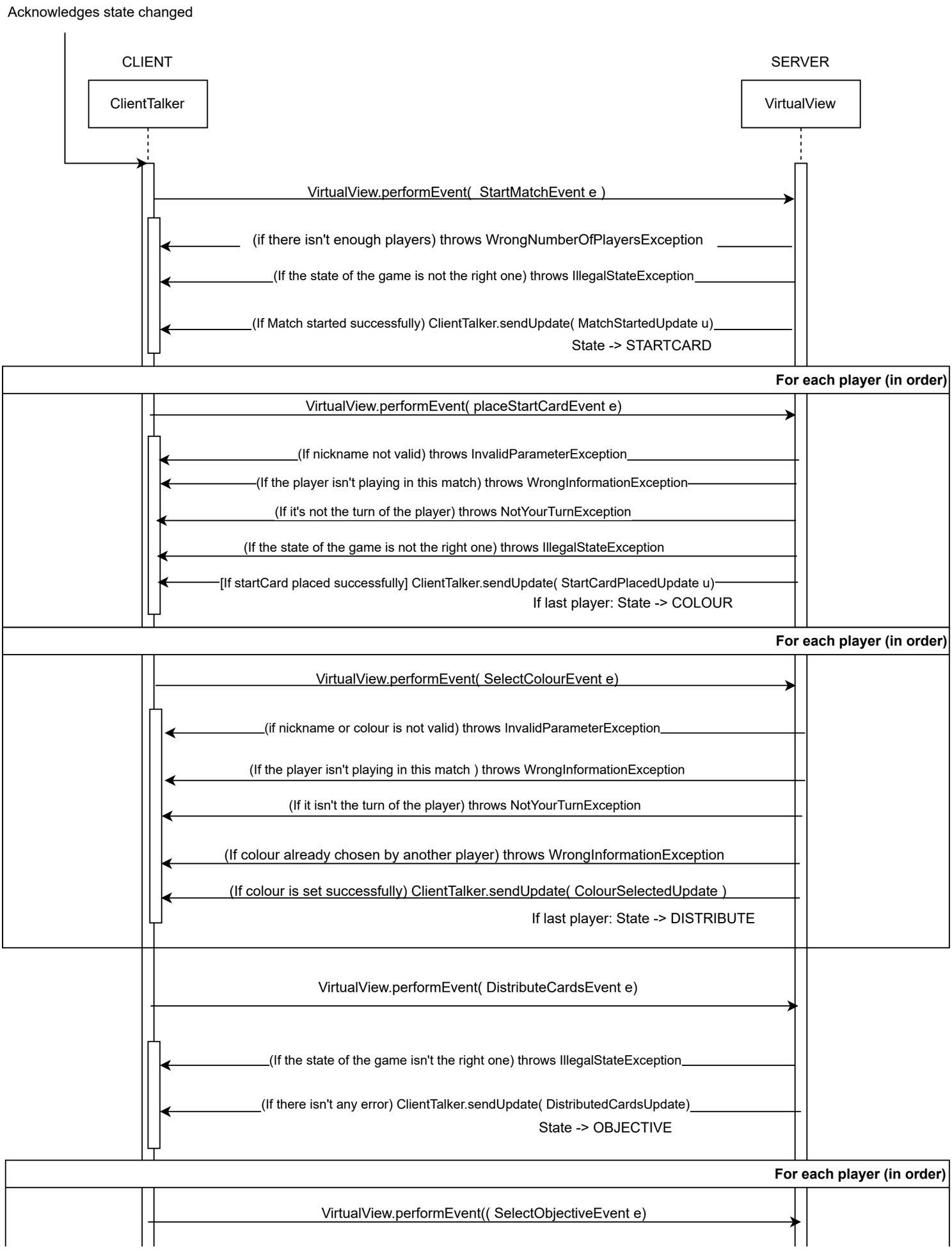
Phase 2: Lobby to be filled (STATE LOBBY)

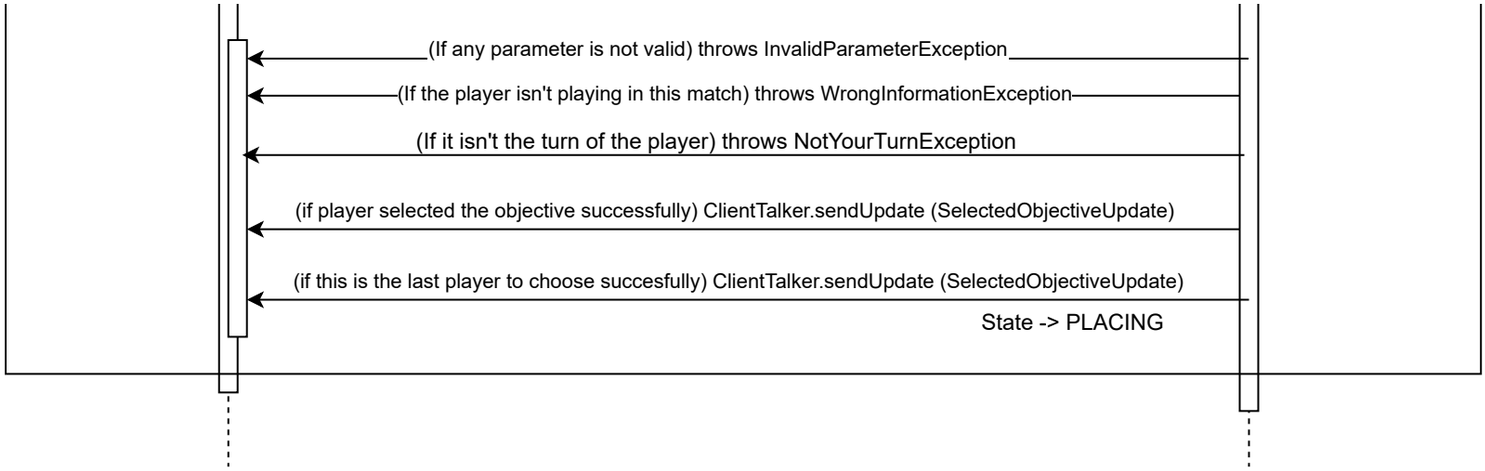
When Match created and/or joined, the client invokes directly the requests from his virtual view in the server (called by his nickname in the register)



Phase 2: Match Initialization

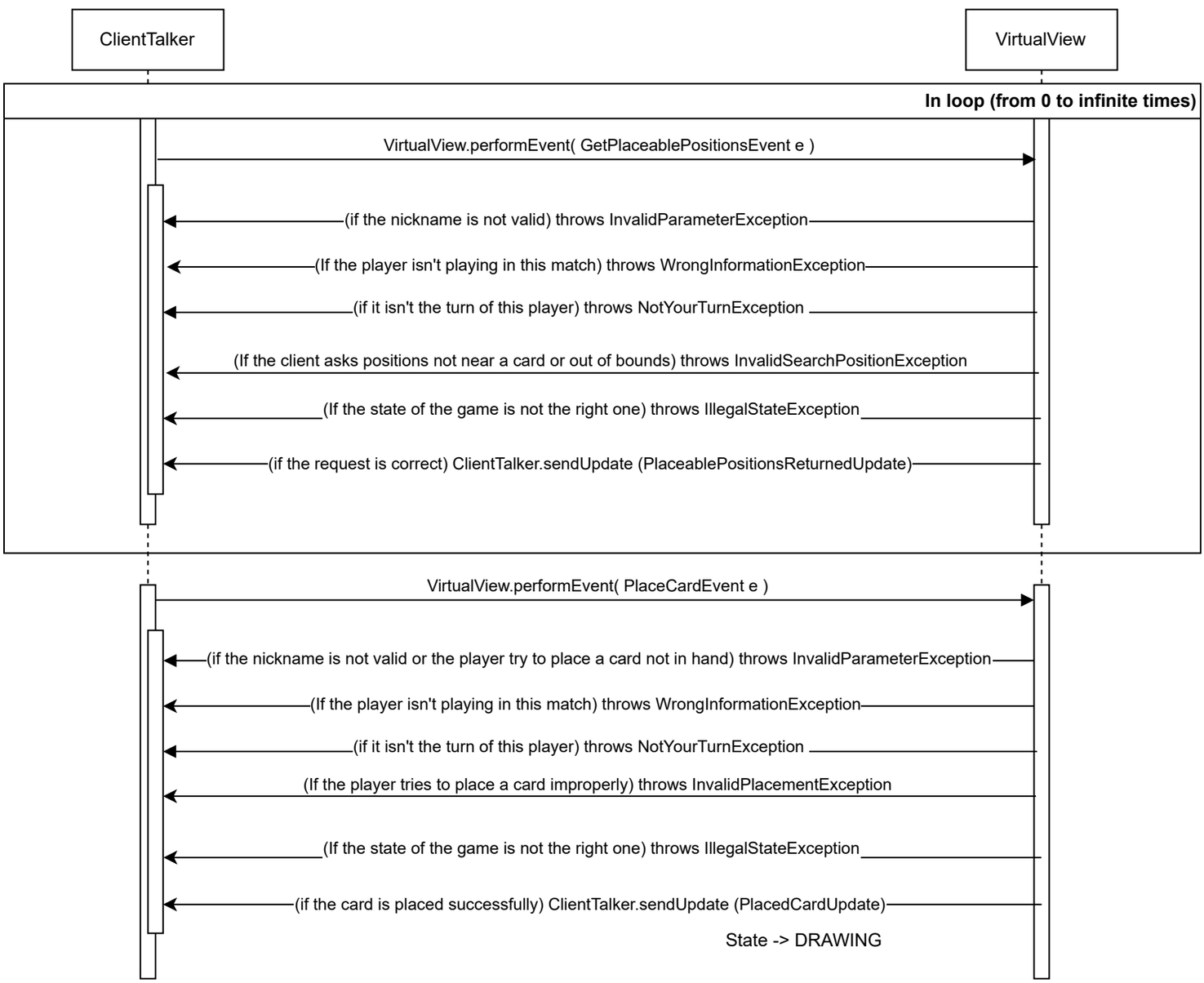
The state is different for every event:
INIZIALIZATION -> STARTCARD -> COLOUR -> DISTRIBUTE -> OBJECTIVE


Acknowledges state changed
CLIENT
SERVER
ClientTalker
VirtualView
VirtualView.performEvent(StartMatchEvent e)
(if there isn't enough players) throws WrongNumberOfPlayersException
(If the state of the game is not the right one) throws IllegalStateException
(If Match started successfully) ClientTalker.sendUpdate(MatchStartedUpdate u)
State -> STARTCARD
For each player (in order)
VirtualView.performEvent(placeStartCardEvent e)
(If nickname not valid) throws InvalidParameterException
(If the player isn't playing in this match) throws WrongInformationException
(If it's not the turn of the player) throws NotYourTurnException
(If the state of the game is not the right one) throws IllegalStateException
[If startCard placed successfully] ClientTalker.sendUpdate(StartCardPlacedUpdate u)
If last player: State -> COLOUR
For each player (in order)
VirtualView.performEvent(SelectColourEvent e)
(if nickname or colour is not valid) throws InvalidParameterException
(If the player isn't playing in this match) throws WrongInformationException
(If it isn't the turn of the player) throws NotYourTurnException
(If colour already chosen by another player) throws WrongInformationException
(If colour is set successfully) ClientTalker.sendUpdate(ColourSelectedUpdate)
If last player: State -> DISTRIBUTE
VirtualView.performEvent(DistributeCardsEvent e)
(If the state of the game isn't the right one) throws IllegalStateException
(If there isn't any error) ClientTalker.sendUpdate(DistributedCardsUpdate)
State -> OBJECTIVE
For each player (in order)
VirtualView.performEvent((SelectObjectiveEvent e)

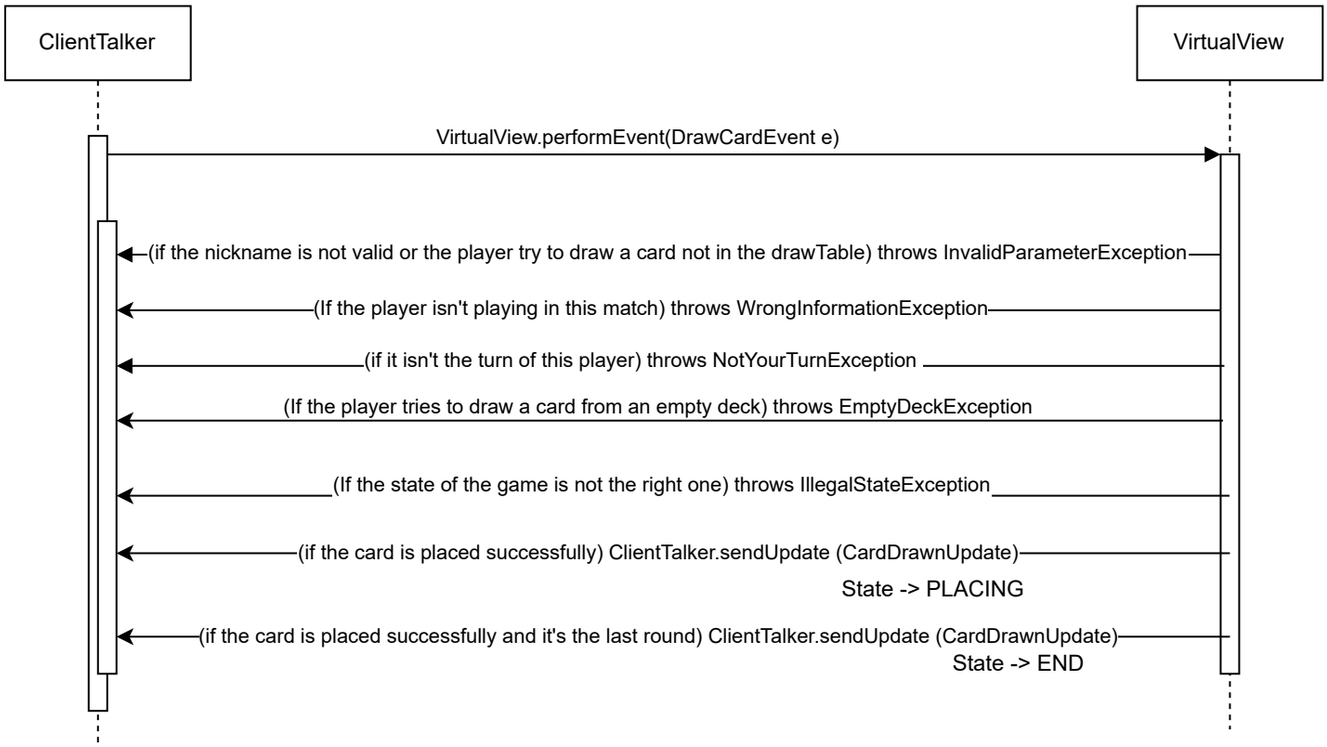


Phase 3: Playing - Place Card Time (STATE PLACING)

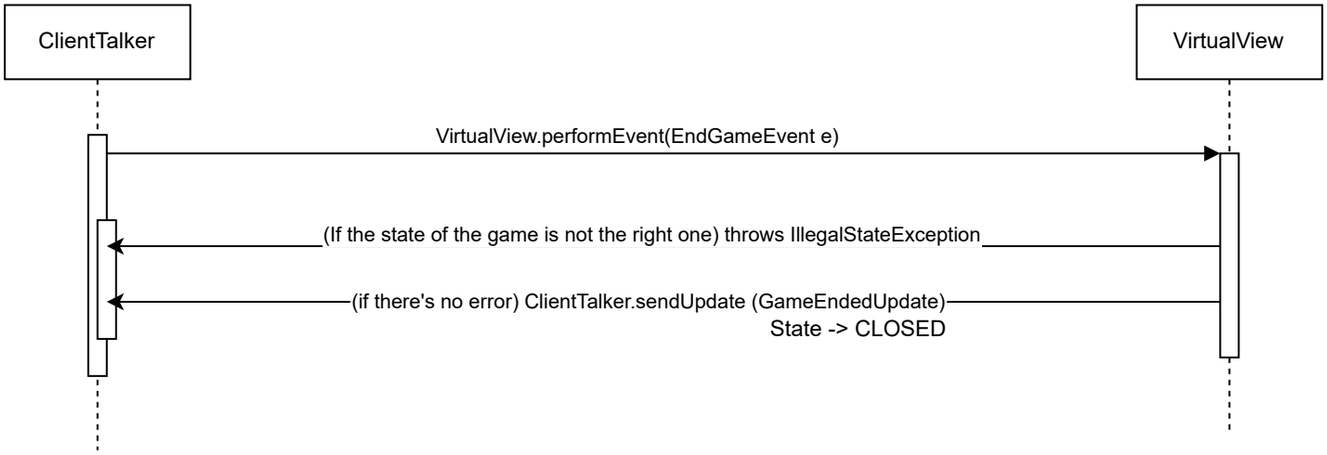
The player who is placing can ask for the possible placeable position near a card infinite times and then ask to place, or ask to place directly



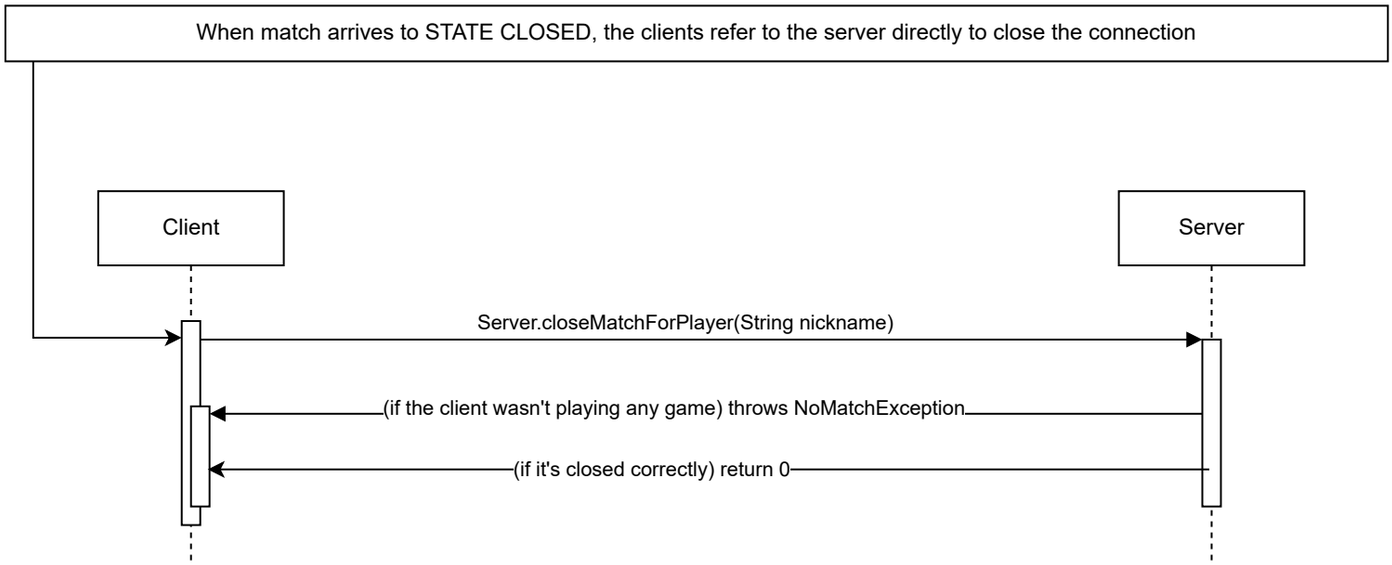
Phase 3: Playing - Draw Card Time (STATE DRAWING)



Phase 4: EndGame (STATE END)

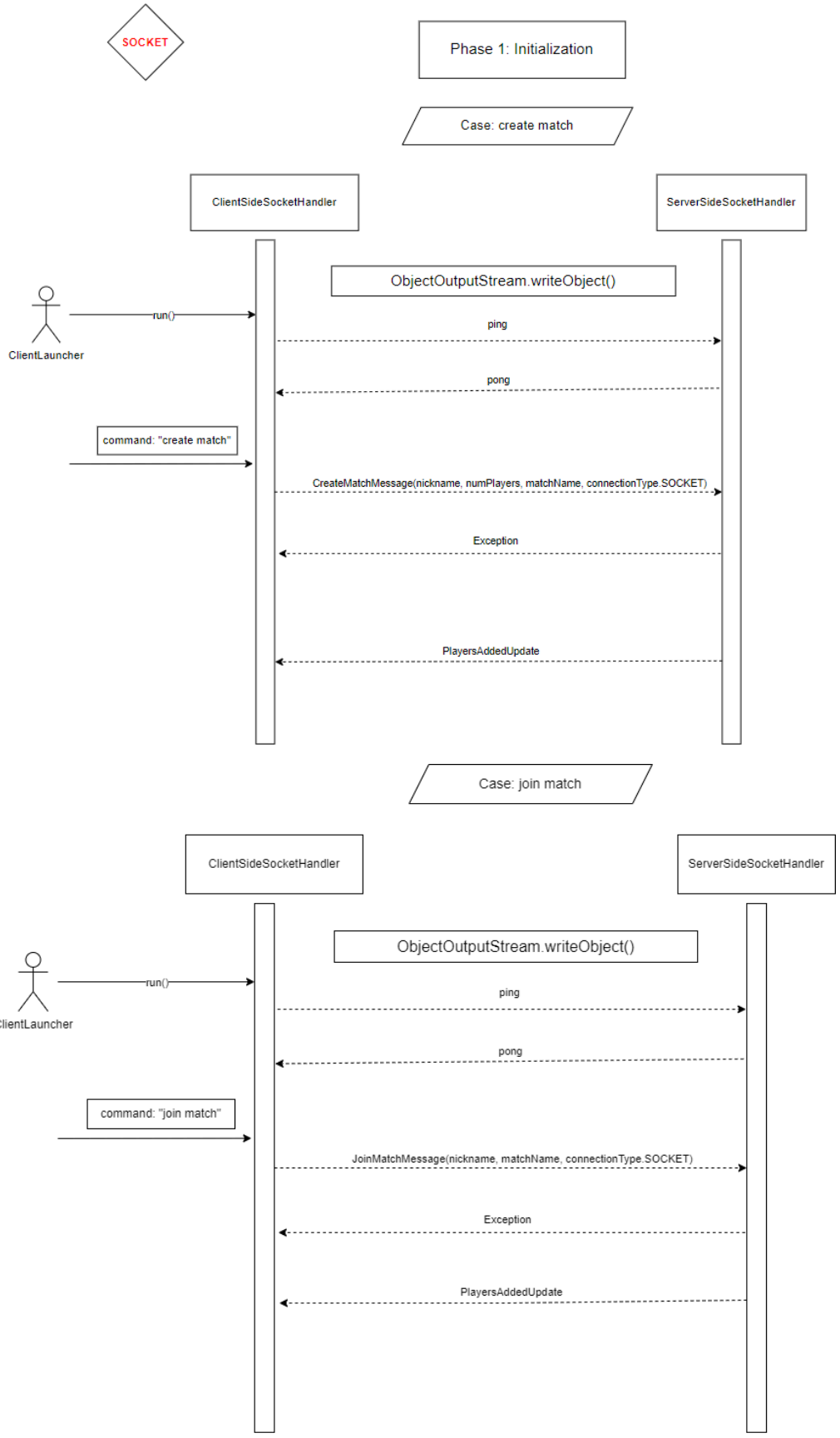


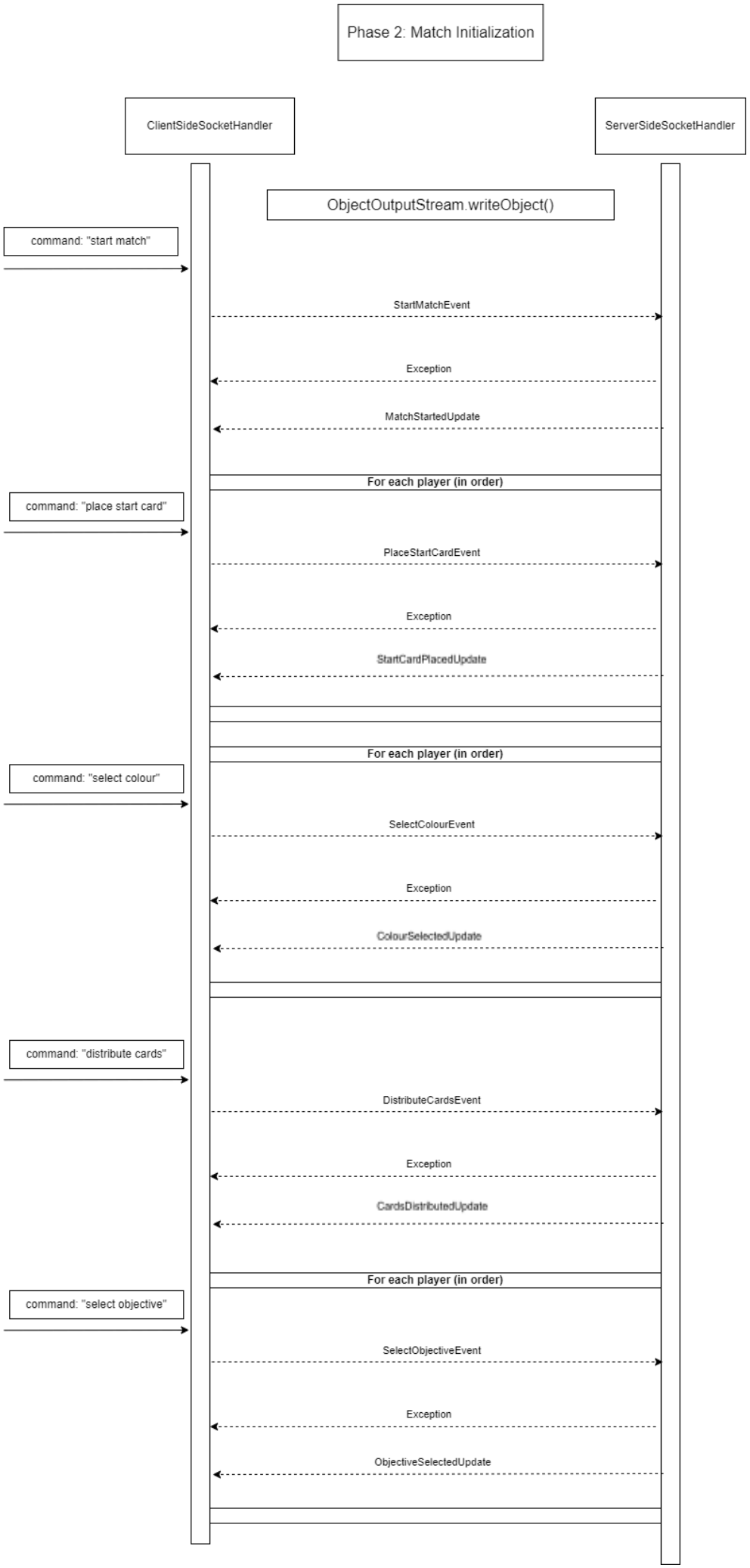
Phase 5: Closing Connection (STATE CLOSED)

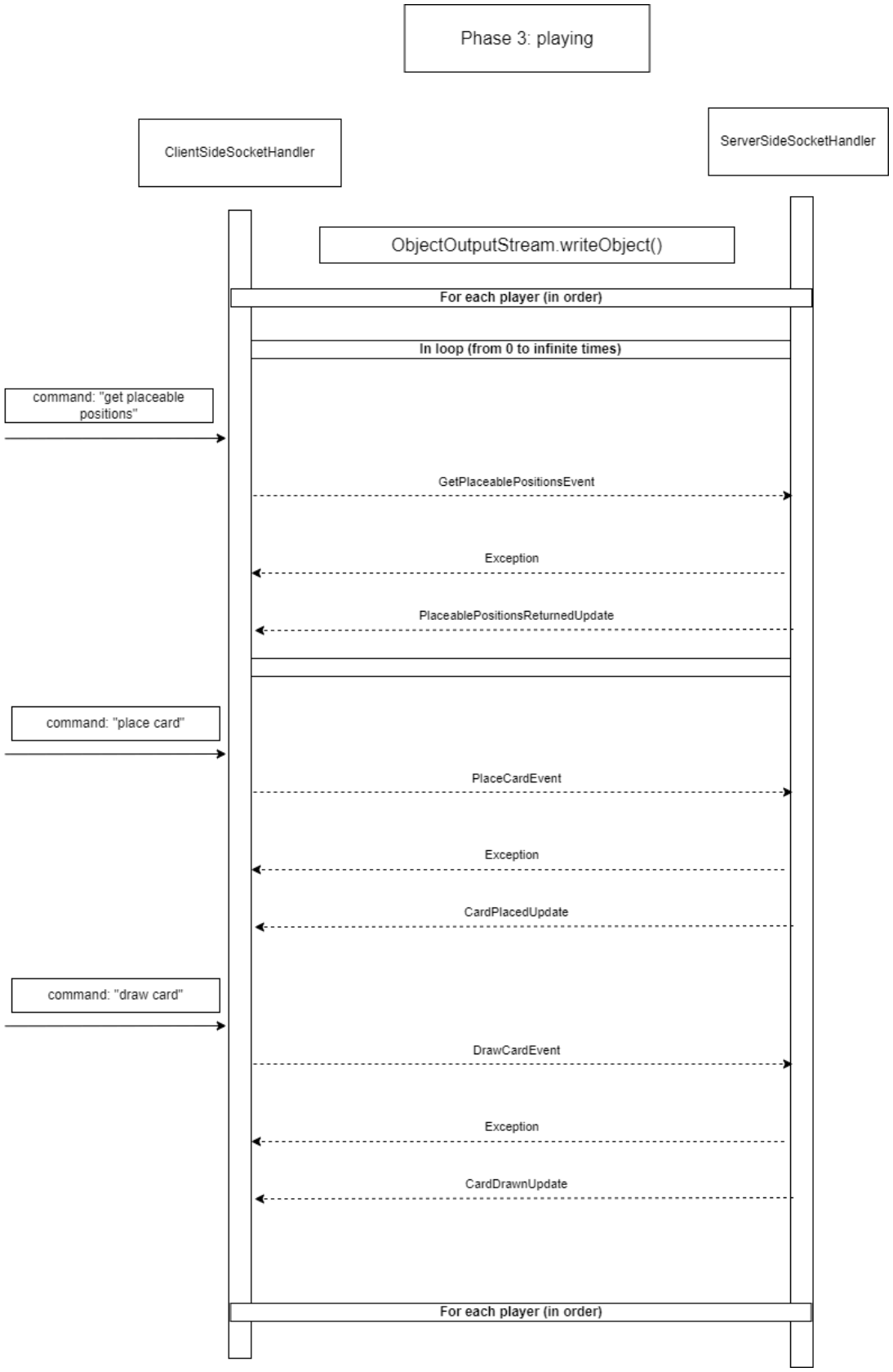


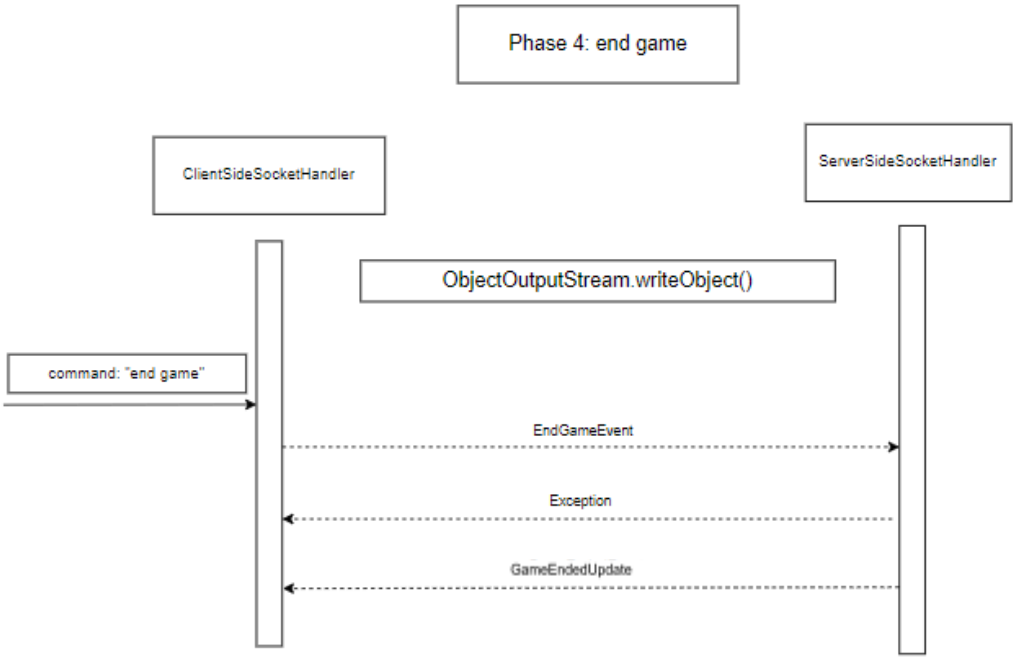
Protocollo Socket

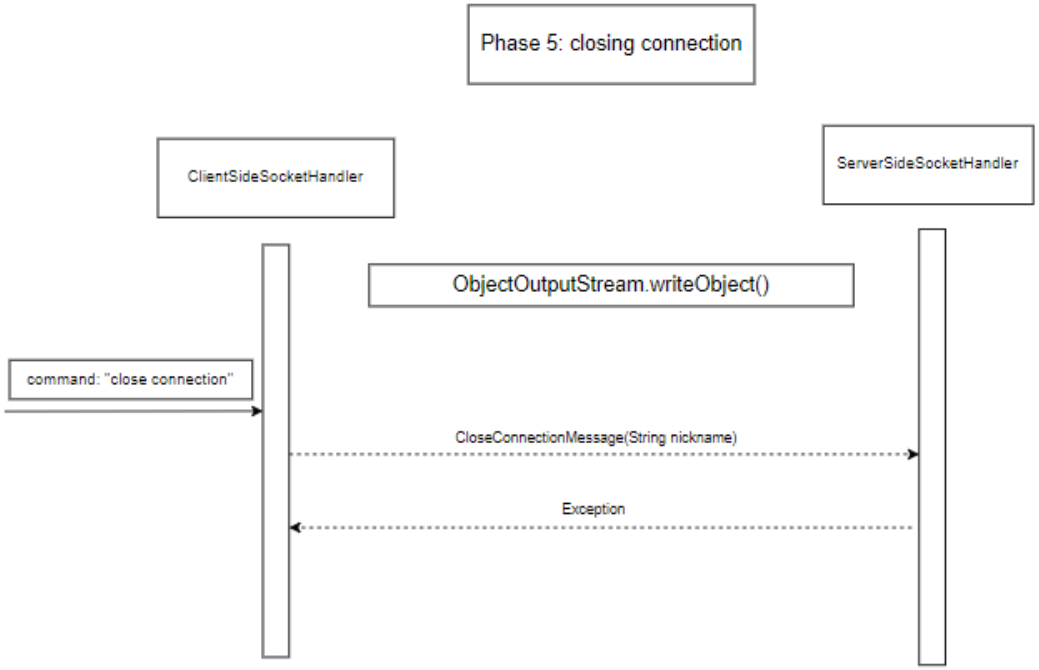
Nei Sequence Diagram seguenti, rappresentiamo il nostro protocollo di comunicazione Socket, indicando quali oggetti vengono scambiati tra client e server.
Per quando riguarda le singole eccezioni (qui rappresentate in generale come Exception), esse sono analoghe a quelle illustrate nel diagramma di RMI.




Phase 2: Match Initialization
ClientSideSocketHandler
ServerSideSocketHandler
ObjectOutputStream.writeObject()
command: "start match"
StartMatchEvent
Exception
MatchStartedUpdate
For each player (in order)
command: "place start card"
PlaceStartCardEvent
Exception
StartCardPlacedUpdate
For each player (in order)
command: "select colour"
SelectColourEvent
Exception
ColourSelectedUpdate
command: "distribute cards"
DistributeCardsEvent
Exception
CardsDistributedUpdate
For each player (in order)
command: "select objective"
SelectObjectiveEvent
Exception
ObjectiveSelectedUpdate

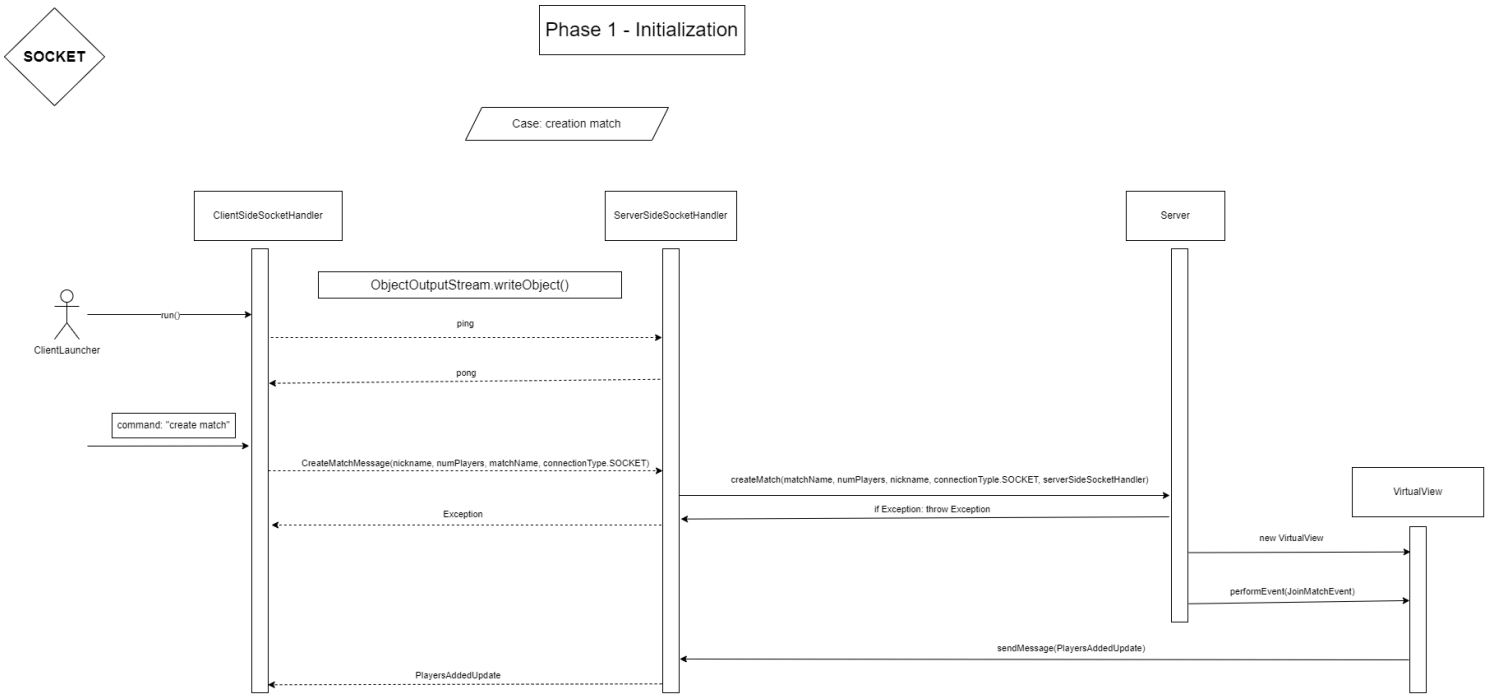

Phase 3: playing
ClientSideSocketHandler
ServerSideSocketHandler
ObjectOutputStream.writeObject()
For each player (in order)
In loop (from 0 to infinite times)
command: "get placeable positions"
GetPlaceablePositionsEvent
Exception
PlaceablePositionsReturnedUpdate
command: "place card"
PlaceCardEvent
Exception
CardPlacedUpdate
command: "draw card"
DrawCardEvent
Exception
CardDrawnUpdate
For each player (in order)


Phase 4: end game
ClientSideSocketHandler
ServerSideSocketHandler
ObjectOutputStream.writeObject()
command: "end game"
EndGameEvent
Exception
GameEndedUpdate

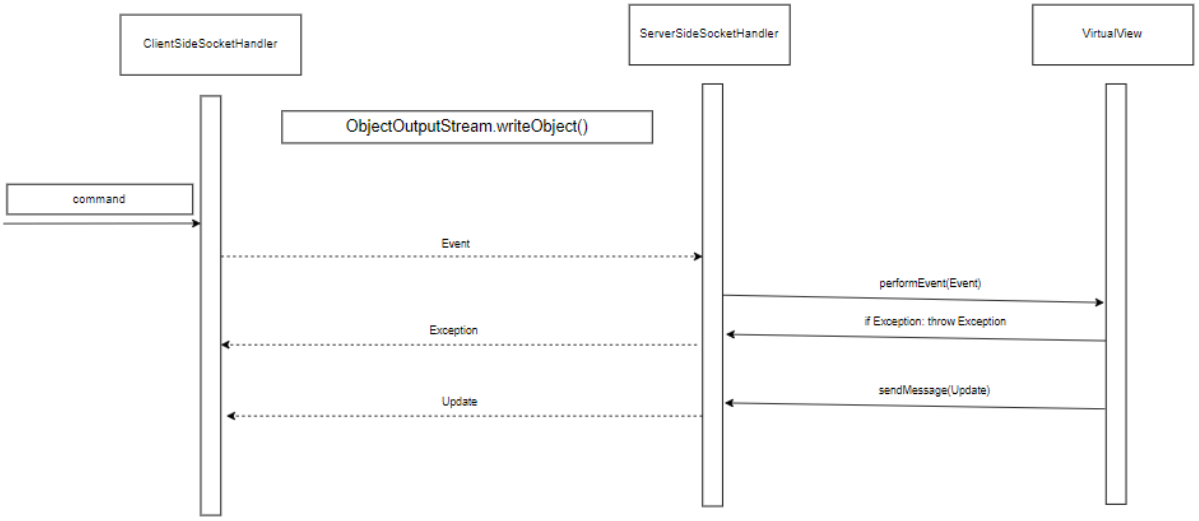

Phase 5: closing connection
ClientSideSocketHandler
ServerSideSocketHandler
ObjectOutputStream.writeObject()
command: "close connection"
CloseConnectionMessage(String nickname)
Exception

Dettaglio interazioni

Tra tutti i casi di interazione illustrati sopra, tre in particolare devono parlare con il server: “create match”, “join match” e “close connection”.
Per mostrare il funzionamento, riportiamo a scopo di esempio il caso “create match”:
il client invia un oggetto di tipo CreateMatchMessage, che ha lo scopo di contenere le informazioni da scambiare. Il ServerSideSocketHandler chiamerà poi un metodo del Server.



In tutti gli altri casi, i messaggi scambiati sono Event (da client a server) e Update (da server a client), e il ServerSideSocketHandler interagisce direttamente con la VirtualView.



Dettaglio Event e Update

Riportiamo nella seguente tabella il contenuto di ciascun Event e Update.

Event	**Update**
JoinMatchEvent: - nickname - virtualView (appena creata dal server, che deve essere aggiunta come Listener del Model)	**PlayersAddedUpdate**: - list of players in the lobby - state (LOBBY or INITIALIZATION)
StartMatchEvent	**MatchStartedUpdate**: - list of players - assigned start cards - colour of the top card of each deck - public cards - player in turn - state (STARTCARD)
PlaceStartCardEvent: - nickname - selected side	**StartCardPlacedUpdate**: - nickname of the player who placed - which card was placed and on which side - player in turn - state (STARTCARD or COLOUR)
SelectColourEvent: - nickname - selected colour	**ColourSelectedUpdate**: - nickname of the player who selected a colour - which colour - player in turn - state (COLOUR or DISTRIBUTION)
DistributeCardsEvent	**CardsDistributedUpdate**: - cards distributed to each player - secret objectives to choose - public objectives - player in turn - state (OBJECTIVE)
SelectObjectiveEvent: - nickname - selected objective	**ColourSelectedUpdate**: - nickname of the player who selected his objective - which objective - player in turn - state (OBJECTIVE or PLACING)
GetPlaceablePositionsEvent: - nickname - coordinates of the card around which the player wants to know where it's possible to place.	**PlaceablePositionsReturnedUpdate**: - nickname of the player who requested the available positions - the available positions - player in turn
PlaceCardEvent: - nickname - index of the card to place - coordinates where to place - selected side	**CardPlacedUpdate**: - nickname of the player who placed - which card and on which side - points gained - player in turn

	- state (DRAWING)
DrawCardEvent: - nickname - index of the deck from which the player wants to draw a card	**CardDrawnUpdate**: - nickname of the player who draws a card - which card - new public card, new colour of the deck - remaining rounds - player in turn - state (PLACING or END)
EndGameEvent: - nickname - index of the deck from which the player wants to draw a card	**GameEndedUpdate**: - number of winners - classification - state (CLOSED)